KCC CODE CLUB · GAME MAKERS

The Long Project

Sky Catcher, week by week

An arcade catch game. Things fall from the sky, you move along the bottom and catch the good ones while dodging the bad ones. Score goes up, lives go down, it gets faster, and at zero lives it's game over.

It is a deliberately unoriginal game, and that is the point. The mechanics are simple enough that an 8-year-old can hold the whole thing in their head, and rich enough that every single concept in the course has an obvious place to live inside it.

Why this project and not something else

- **Playable from week one.** There's never a stretch where they're building infrastructure with nothing to show.
- **Every concept has a natural home.** Loops make things fall. Conditionals make catching work. Variables are the score. Broadcasts are game over. Nothing is bolted on.
- **It demos in 45 seconds.** Critical for the showcase. A parent watching their kid catch falling mangoes gets it immediately. A parent watching a text adventure does not.
- **It's endlessly themeable.** Same code, completely different game depending on the sprites.

Themes — let them choose in week one

Students pick their own theme in class one and stick with it. Suggestions to offer if they're stuck, but push them to invent one:

- Catching mangoes in a basket (bad: coconuts falling on your head)
- A cricket fielder catching balls (bad: no-balls)
- Catching falling stars with a net (bad: asteroids)
- A dog catching bones (bad: cats)
- Catching sadhya dishes onto a banana leaf for Onam (bad: a stray chappal)

The theme is chosen once and never changes. Changing a theme mid-course means redoing sprites and loses a whole session.

Weekly checkpoints

Each row is one class. The game is playable at the end of every row.

Week	Concept taught	What gets added	State at the end of class
1	Sequence, sprites, looks	Player sprite at the bottom, themed backdrop, a greeting	A character that says hello and looks like their idea
2	Events, coordinates	Arrow-key movement, locked to the bottom of the stage	You can drive the player left and right

Week	Concept taught	What gets added	State at the end of class
3	Loops, randomness	One falling object, random x, repeats forever	Things fall from the sky. Looks like a game already
4	Conditionals, sensing	Catching — object resets on contact, plays a sound	**Actually playable.** This is the big week
5	Variables	Score, displayed on stage; a bad object that costs points	It has a score. Kids will now play it obsessively
6	Broadcasts, game state	Lives, title screen, game over screen, restart	A complete game loop with a start and an end
7	Operators, My Blocks	Difficulty ramp, level-up, a rare bonus item	It gets harder as you get better
8	Debugging, UX, sharing	Instructions, credits, sound, playtest fixes, shared online	A finished, shared, demo-able game

Skeleton files — set these up before week one

This is the most important piece of catch-up infrastructure in the program, and it takes about two hours to make once.

Build the reference version of Sky Catcher yourself, saving a separate copy at the end of each week's checkpoint. Publish all eight to a Scratch studio called something like "Game Makers — Skeletons". Then when a child misses week five, you send the parent the week-five skeleton link, they remix it, and the child walks into week six on level ground instead of two weeks behind and quietly humiliated.

Name them plainly: `Sky Catcher — end of Week 3`, and so on.

Do not skip this. Falling behind is the number one reason children drop out of coding clubs, and this single artefact removes it.

Teaching the project build (the last 25 minutes)

Resist the urge to give step-by-step instructions on the projector during project time. The class activity earlier in the session was the guided version; this is where they apply it themselves. Instead:

1. State the goal in one sentence. "By the end of today, things should fall from the sky."
2. Put the *blocks they'll need* on the board, unordered, without the arrangement.
3. Let them struggle for a few minutes before helping. Productive struggle is the lesson.
4. Float. Ask "what did you expect it to do, and what did it do?" rather than fixing their code.

The temptation to grab the mouse and fix it yourself is enormous. Don't. Point at the screen, ask a question, and put your hands behind your back.

Common failure modes and how they present

What the student says	What's usually wrong
"It's not doing anything"	Script isn't attached to a `when green flag clicked` hat block, or it's attached to the wrong sprite
"It only moves once"	Missing a `forever` loop
"It disappears"	Went off-stage; check coordinates, or use `if on edge, bounce`
"It falls too fast to see"	No `wait` inside the loop, or the change in y is too big
"The catching doesn't work"	`touching?` is checking the wrong sprite, or the check isn't inside a `forever` loop
"The score goes up by loads"	The `change score by 1` is inside a `forever` loop and fires every frame — needs a `wait until not touching`
"It worked yesterday"	They're looking at a different copy of the project. Check the project name

That last one is more common than you'd believe. Teach them to name their project properly in week one and check the name at the top of the screen before panicking.

The final showcase version

By week eight the game should have, in this order of priority:

1. A title screen with the game's name and the student's name
2. Instructions — one line, "use the arrow keys to catch the mangoes"
3. Working core loop with score and lives
4. Game over screen with the final score
5. At least one sound
6. A credits line

Anything beyond that is gravy. A polished simple game demos far better than an ambitious broken one, and week eight is for cutting scope, not adding to it.